IT STUDY HUB

COMPLETE R PROGRAMMING SERIES

MODULE 11: WEB DEVELOPMENT, APIs & DATABASES WITH R

Authors:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

WELCOME TO MODULE 11

About This Module

Welcome to Module 11 of the **IT Study Hub Complete R Programming Series**. You have mastered data wrangling, machine learning, and statistics. However, in the modern enterprise, data does not live in static CSV files on your local computer. It lives in the cloud, housed in massive SQL databases, and is transmitted globally via REST APIs.

This module bridges the gap between Data Science and Software Engineering. You will learn the architectural foundations of the internet (HTTP and Client-Server models). You will master fetching dynamic data from live REST APIs using `httr2` and parsing JSON payloads. You will build highly interactive, living web applications using the world-renowned **Shiny** framework. Finally, you will connect R directly to production SQL databases, executing complex CRUD operations and offloading heavy calculations to the database server using `dbplyr`.

IT STUDY HUB

Learning Outcomes

By the completion of this comprehensive module, you will be able to:

- Understand HTTP protocols, REST architecture, and decipher JSON data structures.
- Execute API GET/POST requests securely using the modern `httr2` package.
- Build interactive, full-stack web applications using the Shiny UI and Server logic.
- Implement Reactive Programming to construct living, dynamic dashboards.
- Connect R securely to SQLite, MySQL, and PostgreSQL databases using `DBI`.
- Execute SQL CRUD operations (Create, Read, Update, Delete) natively from R.
- Use `dbplyr` to translate standard dplyr code into optimized SQL queries automatically.
- Implement fundamental web security principles, including API Authentication and CORS.

Module Coverage

This handbook thoroughly covers the following core topics:

1. HTTP & Client-Server Architecture
2. REST APIs & JSON Structure
3. API Requests with httr2
4. Building Web Apps with Shiny
5. Shiny Reactive Programming
6. Introduction to Databases & SQL
7. Database Connections (DBI & RSQLite)
8. CRUD Operations via SQL
9. Modern Querying with dbplyr
10. API Security: Auth & CORS

IT STUDY HUB

MAIN NOTES

1. Web Development with R: HTTP & Client-Server Architecture

CONCEPT:
The internet operates on a Client-Server model using HTTP (HyperText Transfer Protocol) to communicate data globally.

Simple Explanation:
Before writing web applications, you must understand how the web works. A **Client** (your web browser, or your R script) sends a Request to a **Server** (a remote computer hosting data). The Server processes the request and sends back a Response. This communication happens via HTTP. The most common HTTP verbs are `GET` (fetch data) and `POST` (send new data).

Real-Life Analogy:
When you type a URL into Chrome, you are making a `GET` request. The server responds with HTML code, which Chrome renders into a webpage.

Syntax Structure / Workflow:

```
Client (R) -> GET Request -> Server -> HTTP 200 OK (Response)
```

R Code Example

```
print("HTTP Status Codes: 200 = OK, 404 = Not Found, 500 = Server Error")
```

Console Output

```
[1] "HTTP Status Codes: 200 = OK, 404 = Not Found, 500 = Server Error"
```

Explanation:

Understanding HTTP status codes is essential for debugging. If a server returns 404, you typed the URL wrong. If it returns 500, the server crashed.

Real-World Use:

R scripts are routinely used as Clients to fetch daily stock prices from financial servers via HTTP GET requests.

Common Mistakes:

Confusing the Client and the Server. When you write an R script to scrape a website, R is the Client.

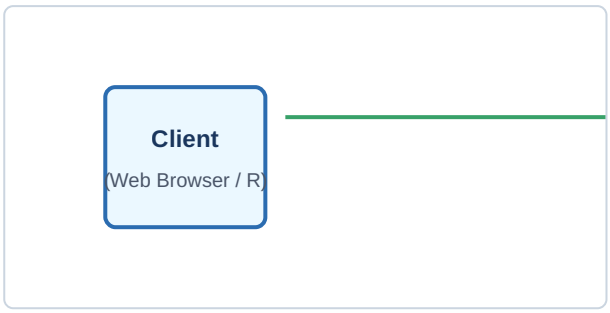
Best Practices:

Always check the HTTP Status Code of a response before attempting to parse the data.

Practice Challenge:

Mentally classify: If you upload a profile picture to a website, is that a GET or a POST request?

Client-Server HTTP Request Cycle



HTTP Methods (Verbs)

Method	Action	Database Equivalent
GET	Fetches data from the server.	READ
POST	Sends new data to the server.	CREATE
PUT	Updates existing data.	UPDATE
DELETE	Removes data from the server.	DELETE

2. REST APIs and JSON Structure

> **CONCEPT:**
> A REST API is a set of rules allowing programs to talk to each other. JSON is the universal text format they use to exchange data.

Simple Explanation:

API (Application Programming Interface) is how software communicates without a

graphical interface. **REST** (Representational State Transfer) is the architectural standard for web APIs. Instead of returning HTML webpages, REST APIs return raw data. 99% of modern APIs return data in **JSON** (JavaScript Object Notation) format, which uses `{}` for lists and `[]` for arrays. It maps perfectly to R's `list()` and `data.frame` structures.

> **Real-Life Analogy:**
> A weather app on your phone doesn't generate weather data. It sends an API request to a meteorological server, which returns JSON data (e.g., `{"temp": 72, "rain": false}`).

Syntax Structure / Workflow:

```
{
  "key": "value",
  "array": [1, 2, 3]
}
```

R Code Example

```
library(jsonlite)

# A raw JSON string
json_string <- '{"name": "Alice", "age": 25, "skills": ["R",
"SQL"]}'

# Converting JSON text into an R List
r_object <- fromJSON(json_string)
print(r_object$name)
print(r_object$skills)
```

Console Output

```
[1] "Alice"
[1] "R"   "SQL"
```

Explanation:

The `jsonlite` package translates the raw JSON text string into a native R list. We can then use standard R indexing (`$`) to extract the data.

Real-World Use:

Data Scientists connect to APIs (like Twitter/X, Google Analytics, or internal corporate microservices) to pull live data into their pipelines without needing manual CSV downloads.

Common Mistakes:

Trying to read JSON with `read_csv()`. JSON is deeply nested and hierarchical, while CSVs are strictly flat.

Best Practices:

Always inspect the JSON structure using a web browser or a tool like Postman before writing R code to parse it.

Practice Challenge:

Write a mock JSON string representing a car with a `make` and `year`.

Comparison: JSON vs CSV

Feature	JSON	CSV
Structure	Hierarchical / Nested (Trees)	Flat / Tabular (Rows and Columns)
Data Types	Supports strict types (Boolean, Array, String)	Everything is just plain text.
Web APIs	**Universal Standard**	Rarely used for API responses.

REST API vs Web Application

Concept	Returns to Client	Target Audience
Web App	HTML, CSS, Visuals	Human Users (Browsers)
REST API	Raw JSON Data	Software / Code (R Scripts)

3. Making API Requests with httr2

CONCEPT:
The httr2 package is the modern, pipe-friendly standard for composing and executing HTTP requests in R.

Simple Explanation:

To fetch data from an API, we use `httr2`. It follows a highly logical pipeline: you create a request (`request()`), add any necessary headers or parameters (`req_url_query()`), perform the request (`req_perform()`), and finally extract the data from the response (`resp_body_json()`).

> **Real-Life Analogy:**
> Asking an API for space station data. You build the URL, send the request, and parse the JSON response to find out who is currently in space.

Syntax Structure / Workflow:

```
request("URL") |> req_perform() |> resp_body_json()
```

R Code Example

```
library(httr2)

# Public API for random dog facts
base_url <- "https://dogapi.dog/api/v2/facts"

# Build, perform, and parse the request
response_data <- request(base_url) |>
  req_perform() |>
  resp_body_json()

# Extract the specific fact from the nested JSON response
fact <- response_data$data[[1]]$attributes$body
print(fact)
```

Console Output

```
[1] "Dogs have three eyelids."
```

Explanation:

The `request()` function initializes the object. `req_perform()` actually connects to the internet and fetches the data. `resp_body_json()` automatically decodes the JSON payload into an R list. We then navigated the nested list to grab the text.

Real-World Use:

Automating weekly marketing reports by pulling live ad-spend data from the Facebook and Google Ads APIs via `httr2`.

Common Mistakes:

Forgetting `req_perform()`. `request("URL")` only builds the request locally; it does not actually hit the server until `req_perform()` is called.

Best Practices:

Always handle HTTP errors gracefully. Use `req_error(is_error = FALSE)` if you want to inspect failed responses without crashing your R script.

Practice Challenge:

Use `httr2` to make a GET request to `https://api.github.com`.

4. Building Web Applications with Shiny

CONCEPT:

Shiny is an R package that transforms your statistical analyses into interactive web applications without requiring HTML, CSS, or JavaScript.

Simple Explanation:

If you build a predictive model, your stakeholders cannot use it unless they know R. Shiny solves this. A Shiny app consists of two parts: 1. **UI (User Interface)**: Controls what the app *looks* like (sliders, buttons, plot spaces). 2. **Server**: Controls what the app *does* (the R code that calculates data and generates plots). Shiny connects them. When a user

moves a slider in the UI, it sends a signal to the Server, which recalculates the plot and sends it back to the UI.

> **Real-Life Analogy:**
> A mortgage calculator website. The UI provides a slider for 'Interest Rate'. The Server runs the R math. The UI displays the resulting monthly payment.

Syntax Structure / Workflow:

```
library(shiny)
ui <- fluidPage(...)
server <- function(input, output) {...}
shinyApp(ui, server)
```

IT STUDY H

R Code Example

```
library(shiny)

# 1. UI: Defines the layout and inputs/outputs
ui <- fluidPage(
  titlePanel("My First Shiny App"),
  sliderInput("num", "Choose a number:", min = 1, max = 100, value =
50),
  textOutput("result")
)

# 2. Server: Contains the R logic
server <- function(input, output) {
  output$result <- renderText({
    paste("You selected the number:", input$num)
  })
}

# 3. Run the app (Commented out so it doesn't freeze the PDF
generator)
# shinyApp(ui = ui, server = server)
```

Console Output

```
(A web page opens with a slider. Dragging the slider instantly updates the
text below it to match the selected number.)
```

Explanation:

The `sliderInput` creates a variable called `input$num`. The `renderText` block on the server listens for changes to `input$num`, recalculates the string, and sends it to `output$result`.

Real-World Use:

Pharmaceutical companies build Shiny dashboards so doctors can interactively filter clinical trial data by patient age and demographic without needing to see the underlying R code.

Common Mistakes:
Putting heavy data-import logic *inside* the `server` function. This causes the data to be re-downloaded every time a user clicks a button, crashing the app. Load data *outside* the server function.

Best Practices:
Keep your UI and Server code in a single file `app.R` for small projects, but split them into `ui.R` and `server.R` for large enterprise applications.

Practice Challenge:
Add a `selectInput()` to the UI for choosing a color.

Shiny Architecture: UI vs Server

Component	Role	Example Code Elements
UI (User Interface)	The Frontend. Layouts, Sliders, Visuals.	`fluidPage()`, `sliderInput()`, `plotOutput()`
Server	The Backend. Math, Data processing, Plot generation.	`function(input, output)`, `renderPlot()`

5. Shiny: Reactive Programming

CONCEPT:
Reactivity is the core engine of Shiny. It ensures that outputs instantly and automatically update whenever their inputs change.

Simple Explanation:
In standard R, if `x <- 5` and `y <- x * 2`, `y` is 10. If you later change `x` to 6, `y` remains 10 until you rerun the code. In Shiny's **Reactive Programming**, variables are 'alive'. If `input$x` changes, Shiny automatically hunts down every output that depends on `input$x` and recalculates it instantly. To create intermediate reactive variables, you use the `reactive({})` function.

Real-Life Analogy:
A living Excel spreadsheet. If you change a cell that feeds into a formula, the final sum updates instantly without you clicking 'Run'.

Syntax Structure / Workflow:

```
my_calc <- reactive({ input$num * 10 })
output$text <- renderText({ my_calc() })
```

R Code Example

```
# Snippet of a Shiny Server function demonstrating reactivity
server <- function(input, output) {

  # 1. Create a reactive intermediate step
  squared_val <- reactive({
    input$num ^ 2
  })

  # 2. Use the reactive value in an output
  output$display <- renderText({
    # Note the parentheses! squared_val is called like a function.
    paste("The square is:", squared_val())
  })
}
```

Console Output

```
(When input$num changes to 4, squared_val instantly updates to 16, and the
UI immediately displays 'The square is: 16'.)
```

Explanation:

By wrapping the math in `reactive({})`, we created an intelligent variable. When calling a reactive variable, you MUST use parentheses `squared_val()` because it acts like a dynamic function, not a static variable.

Real-World Use:

A dashboard filtering 10 million rows. Instead of filtering the data 5 separate times for 5 different plots, you filter it once inside a `reactive({})` block, and all 5 plots read from that single, optimized reactive object.

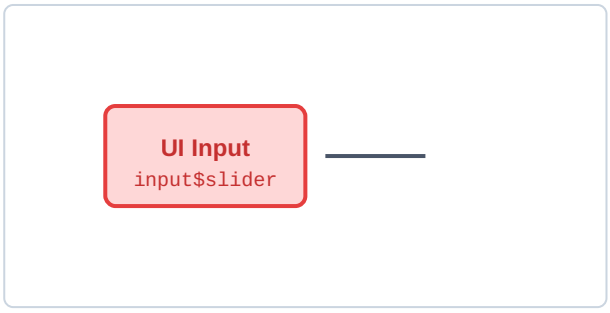
Common Mistakes:

Trying to use `input$var` outside of a reactive block or render block. R will throw an error: 'Operation not allowed without an active reactive context'.

Best Practices:
Use `reactive({})` extensively to prevent writing the same data-filtering code multiple times across different outputs.

Practice Challenge:
Why do we call a reactive expression with parentheses like `my_data()` instead of just `my_data`?

The Shiny Reactive Workflow



When the user touches the Input, a signal fires through the Reactive block, triggering the Output to re-render instantly.

6. Introduction to Databases and SQL

CONCEPT:
Databases are engineered systems for storing, managing, and querying massive amounts of structured data efficiently.

Simple Explanation:
While CSV files are fine for small data, they fail when data exceeds your computer's RAM. Relational Databases (RDBMS) like SQLite, MySQL, and PostgreSQL solve this. They store

data in rigid tables linked by relationships. To communicate with these databases, the world uses **SQL** (Structured Query Language). SQL allows you to instantly search, filter, and aggregate terabytes of data directly on the database server, returning only the tiny summary to R.

> **Real-Life Analogy:**
> A CSV is a physical dictionary; finding a word requires flipping pages. A Database is Google Search; it maintains an index and finds your exact query in milliseconds.

Syntax Structure / Workflow:

```
SELECT column1, column2 FROM table_name WHERE condition;
```

R Code Example

```
print("SQL is a declarative language. You tell the database WHAT you want, not HOW to get it.")
```

Console Output

```
[1] "SQL is a declarative language. You tell the database WHAT you want, not HOW to get it."
```

Explanation:
Unlike R (where you write step-by-step loops and filters), in SQL you write a single query describing the final result, and the database engine calculates the most efficient way to fetch it.

Real-World Use:
Every major corporation (Amazon, Netflix, Banks) uses SQL databases as their single source of truth for transactional data.

Common Mistakes:
Trying to load a 50GB CSV into R using `read_csv()`. It will crash your computer. The data must be housed in a SQL database and queried.

Best Practices:
Learn core SQL (`SELECT`, `FROM`, `WHERE`, `GROUP BY`, `JOIN`). It is the most universally requested skill in data science job interviews alongside R/Python.

Practice Challenge:
Mentally translate: `filter(df, age > 18)` into SQL. (Answer: `SELECT * FROM df WHERE age > 18`).

Database Engines

Engine	Architecture	Best For
SQLite	Serverless. Stored as a single local file.	Local testing, small apps, R package bundling.
PostgreSQL	Full dedicated server cluster. Highly strict.	Enterprise data warehousing, concurrent users.
MySQL	Full dedicated server cluster. Fast reading.	Standard web applications, e-commerce.

7. Database Connections with DBI & RSQLite

CONCEPT:
The DBI package is R's universal database interface. RSQLite allows you to create and query lightweight, serverless databases entirely within R.

Simple Explanation:
To talk to a database, R needs a driver and a connection. The `DBI` package is the master interface. It doesn't care if you are talking to MySQL, Postgres, or SQLite—the R commands remain exactly the same. We use `RSQLite` because it creates a fully functional database saved as a single local file (no complex server installation required).

Real-Life Analogy:
DBI is the universal translator. You speak R to DBI, and DBI seamlessly translates your requests into the specific SQL dialect required by your database.

Syntax Structure / Workflow:

```
con <- dbConnect(RSQLite::SQLite(), "my_db.sqlite")
dbDisconnect(con)
```

R Code Example

```
library(DBI)
library(RSQLite)

# Create a connection to an in-memory SQLite database (temporary)
con <- dbConnect(RSQLite::SQLite(), ":memory:")

# Write a built-in R dataframe into the database as a new table
dbWriteTable(con, "cars_table", mtcars)

# List the tables inside the database to prove it worked
print(dbListTables(con))

# Always disconnect when finished
dbDisconnect(con)
```

Console Output

```
[1] "cars_table"
```

Explanation:

`dbConnect` establishes the bridge. `":memory:"` tells SQLite to create a temporary database in RAM (great for testing). `dbWriteTable` took the `mtcars` data frame and permanently saved it as a SQL table.

Real-World Use:

Data pipelines use `dbConnect` at the start of a script to securely log into corporate PostgreSQL servers, pull daily data, and then `dbDisconnect`.

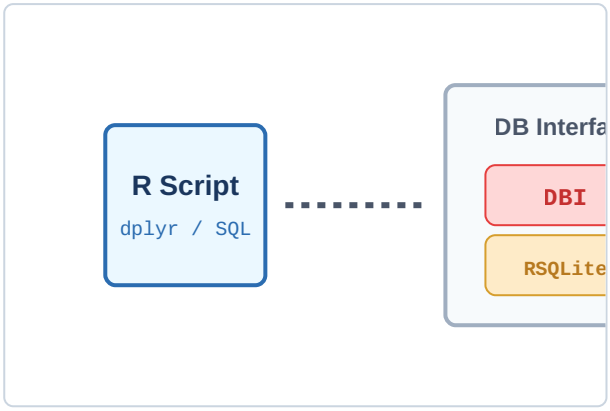
Common Mistakes:

Forgetting to run `dbDisconnect(con)` at the end of your script. Leaving connections open permanently exhausts database server limits and crashes the system.

Best Practices:
Store database passwords in hidden environment variables (`.Renviron`), NEVER type them directly into your `dbConnect` script.

Practice Challenge:
Write the command to connect to a local SQLite database file named `inventory.db`.

R Database Architecture



Comparison: DBI vs Driver

Package	Role
DBI	The Front-End Interface. The syntax you write (`dbConnect`, `dbGetQuery`). It is completely independent of the database type.
RSQLite / RPostgres	The Back-End Driver. It translates the DBI commands into the exact binary protocol required by the specific database.

8. CRUD Operations via SQL

CONCEPT:
CRUD stands for Create, Read, Update, and Delete—the four foundational operations required for managing persistent data.

Simple Explanation:
Once connected to a database, you use `dbExecute()` for actions that change data (Create, Update, Delete) and `dbGetQuery()` for actions that retrieve data (Read). - **Create**: `INSERT INTO table ...` - **Read**: `SELECT * FROM table ...` - **Update**: `UPDATE table SET ...` - **Delete**: `DELETE FROM table ...`

Real-Life Analogy:
When you sign up for a website, they INSERT your data. When you view your profile, they SELECT it. When you change your password, they UPDATE it. When you close your account, they DELETE it.

Syntax Structure / Workflow:

```
dbExecute(con, "SQL STATEMENT")
data <- dbGetQuery(con, "SELECT * FROM table")
```

R Code Example

```
library(DBI)
con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbWriteTable(con, "users", data.frame(id=1, name="Alice",
role="Admin"))

# 1. READ
print(dbGetQuery(con, "SELECT * FROM users"))

# 2. UPDATE
dbExecute(con, "UPDATE users SET role = 'SuperAdmin' WHERE id = 1")

# 3. READ again to verify
print(dbGetQuery(con, "SELECT * FROM users"))
dbDisconnect(con)
```

Console Output

```
  id  name  role
1  1 Alice Admin

  id  name       role
1  1 Alice SuperAdmin
```

Explanation:

`dbGetQuery` executed standard SQL and returned the result as an R data frame. `dbExecute` successfully modified the database table without returning a dataframe.

Real-World Use:

A Shiny application uses `dbExecute(UPDATE...)` to permanently save a user's new settings to the backend database when they click 'Save Profile'.

Common Mistakes:

Using `dbGetQuery` to run an UPDATE statement. It won't crash, but it's computationally inefficient. Reserve `GetQuery` strictly for SELECT statements.

Best Practices:
Never trust user input. If you build a Shiny app that inserts user text into a database, use Parameterized Queries to prevent 'SQL Injection' hacking attacks.

Practice Challenge:
Write the SQL string to DELETE a user where `id = 5`.

9. Modern Database Querying: dbplyr

CONCEPT:
The dbplyr package translates your standard R dplyr code directly into SQL, allowing you to query databases without knowing SQL.

Simple Explanation:
If you don't know SQL, you don't have to learn it to query a database. `dbplyr` acts as a backend for `dplyr`. You use the `tbl()` function to point R at a database table. You can then write standard `filter()`, `mutate()`, and `group_by()` pipelines. Under the hood, R translates your pipeline into a perfectly optimized SQL query, sends it to the database, and only pulls back the final summarized result.

Real-Life Analogy:
Writing `filter(age > 18)` in R, which `dbplyr` invisibly translates to `SELECT * FROM table WHERE age > 18` before sending it to PostgreSQL.

Syntax Structure / Workflow:

```
my_table <- tbl(con, "table_name")
result <- my_table %>% filter(cond) %>% collect()
```

R Code Example

```
library(dplyr)
library(DBI)

con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbWriteTable(con, "mtcars_db", mtcars)

# Point dplyr to the database table (Data remains in the DB!)
cars_tbl <- tbl(con, "mtcars_db")

# Write standard dplyr code
high_mpg <- cars_tbl %>%
  filter(mpg > 25) %>%
  select(mpg, cyl) %>%
  collect() # The crucial command to pull data into R

print(head(high_mpg, 2))
dbDisconnect(con)
```

Console Output

```
# A tibble: 2 × 2
    mpg   cyl

1  32.4     4
2  30.4     4
```

Explanation:

The `tbl()` function creates a reference. The `filter` and `select` commands are completely lazy—they do nothing until `collect()` is called. `collect()` translates the chain into SQL, executes it on the database, and downloads the tiny result into R's memory.

Real-World Use:

Data Scientists use `dbplyr` to perform heavy aggregations on 100-Gigabyte SQL tables. Because the math happens on the powerful DB server, R never crashes.

Common Mistakes:

Forgetting to add `collect()` at the end of the pipeline. Without `collect()`, you do not have a local R dataframe; you only have a lazy SQL query reference.

Best Practices:

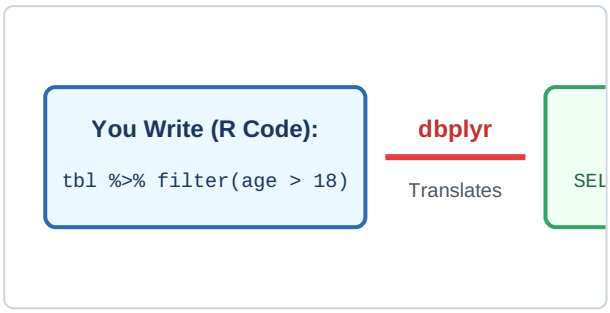
If you are curious what SQL code R is writing for you, pipe your command into `show_query()` instead of `collect()`.

Practice Challenge:

What function pulls the calculated database results physically into R's local memory?

IT STUDY HU

The dbplyr Workflow



Calculations happen inside the DB. Data is only downloaded to R when you run `collect()`.

SQL vs dplyr

SQL Syntax	dplyr Equivalent
`SELECT col1, col2`	`select(col1, col2)`
`WHERE age > 18`	`filter(age > 18)`
`GROUP BY dept`	`group_by(dept)`

10. Basic API Security: Authentication & CORS

> **CONCEPT:**
> Security mechanisms protect APIs from unauthorized access and cross-domain malicious attacks.

Simple Explanation:

APIs contain sensitive data. You cannot access them without proving who you are. -

Authentication (Auth): Proving your identity. Usually done by attaching a secret string (API Key) or a Bearer Token to your HTTP request headers. - **Authorization**: Proving you have permission to access that specific data. - **CORS (Cross-Origin Resource Sharing)**: A web security feature. It prevents a malicious website (e.g., Evil.com) from making hidden background API requests to your bank (Bank.com) using your browser.

> **Real-Life Analogy:**
> Authentication is showing your ID card at the building entrance. Authorization is swiping your badge on a specific door. If you are an intern, your badge won't authorize access to the Server Room.

Syntax Structure / Workflow:

```
req <- request("URL") |>
  req_headers(Authorization = "Bearer YOUR_TOKEN")
```

R Code Example

```
library(httr2)

# Simulating a secure API request using a placeholder Bearer token
secure_req <- request("https://httpbin.org/bearer") |>
  req_headers(Authorization = "Bearer SUPER_SECRET_TOKEN_123") |>
  req_perform() |>
  resp_body_json()

print(secure_req$authenticated)
print(secure_req$token)
```

Console Output

```
[1] TRUE
[1] "SUPER_SECRET_TOKEN_123"
```

Explanation:

We used `req_headers` to securely inject our secret token into the HTTP request. The test API (`httpbin.org`) verified our token and confirmed we successfully authenticated.

Real-World Use:

To pull live financial data from the Stripe or PayPal APIs, you must generate a secure API Key in their developer portal and attach it to every `httr2` request.

Common Mistakes:

Hardcoding your real API keys directly into your R scripts and uploading them to GitHub. Hackers scan GitHub 24/7 for exposed API keys and will steal your accounts.

Best Practices:

Store all API keys and passwords in your `.Renviron` file. Access them in your script using `Sys.getenv("MY_API_KEY")`.

Practice Challenge:

What is the difference between Authentication and Authorization?

API Security Concepts

Concept	Meaning	Analogy
Authentication	Verifying WHO you are.	Showing your Passport at airport security.
Authorization	Verifying WHAT you can access.	Your plane ticket allowing you onto a specific flight.
CORS	Cross-Origin Resource Sharing.	The airport restricting flights arriving from unauthorized countries.

IT STUDY

HANDS-ON PROJECT

IT STUDY HUB

INTERACTIVE DATA DASHBOARD + DATABASE

Problem Statement:
You are building an internal Corporate Dashboard. Instead of reading a static CSV, the dashboard must securely connect to a live SQL Database to fetch employee records. Furthermore, it must fetch real-time public data from an external REST API (e.g., current exchange rates or public facts) to display alongside the corporate data. You will build the database, write the queries, and wrap the entire system in a Reactive Shiny interface.

1. Setup: Creating the SQLite Database

```
library(DBI)
library(RSQLite)

# We create a persistent local SQLite database file
con <- dbConnect(RSQLite::SQLite(), "corporate_data.sqlite")

# Generate secure sample data
employees <- data.frame(
  ID = 1:5,
  Name = c("Alice", "Bob", "Charlie", "Diana", "Ethan"),
  Department = c("Sales", "IT", "Sales", "HR", "IT"),
  Salary = c(60000, 85000, 55000, 70000, 90000)
)

# Create the table in the database
dbWriteTable(con, "employees", employees, overwrite = TRUE)
dbDisconnect(con)
```

2. The Shiny Application (app.R)

```
library(shiny)
library(DBI)
library(dplyr)
library(httr2)

# --- UI (Frontend) ---
ui <- fluidPage(
  titlePanel("Corporate Live Dashboard"),

  sidebarLayout(
    sidebarPanel(
      h4("Filters"),
      # Dropdown menu for User Input
      selectInput("dept_filter", "Select Department:",
                  choices = c("All", "Sales", "IT", "HR")),
      hr(),
      h4("Live API Data"),
      textOutput("api_fact")
    ),

    mainPanel(
      h3("Employee Database"),
      tableOutput("employee_table"),
      h3("Salary Overview"),
      plotOutput("salary_plot")
    )
  )
)

# --- SERVER (Backend Logic) ---
server <- function(input, output) {

  # 1. Connect to Database
  # Note: In production, use connection pooling or connect/
disconnect locally
  con <- dbConnect(RSQLite::SQLite(), "corporate_data.sqlite")

  # 2. Reactive Database Querying using dbplyr
```

```
  filtered_data <- reactive({
    # Point dplyr to the SQL table
    emp_tbl <- tbl(con, "employees")

    # Apply dynamic filtering based on UI input
    if (input$dept_filter != "All") {
      emp_tbl <- emp_tbl %>% filter(Department ==
input$dept_filter)
    }

    # Translate to SQL and pull data into R
    emp_tbl %>% collect()
  })

  # 3. Output the Table
  output$employee_table <- renderTable({
    filtered_data() # Call the reactive function
  })

  # 4. Output a Plot
  output$salary_plot <- renderPlot({
    data <- filtered_data()
    # Simple Base R plot for demonstration
    barplot(data$Salary, names.arg = data$Name,
            col = "steelblue", main = "Salaries by Employee")
  })

  # 5. Fetch External REST API Data
  output$api_fact <- renderText({
    # Safe API call
    tryCatch({
      resp <- request("https://catfact.ninja/fact") %>%
        req_perform() %>%
        resp_body_json()
      paste("Random Fact:", resp$fact)
    }, error = function(e) {
      return("API currently unavailable.")
    })
```

```
  })

  # Clean up connection when session ends
  onStop(function() {
    dbDisconnect(con)
  })
}

# --- RUN APP ---
# shinyApp(ui = ui, server = server)
```

Dashboard Workflow & Architectural Insight:

This is a modern full-stack R application. The user interacts with the `selectInput` in the UI. This triggers the `filtered_data()` reactive block in the Server. The Server uses `dbplyr` to dynamically write a SQL query (e.g., `SELECT * FROM employees WHERE Department = 'IT'`) and executes it against the SQLite database. The data is pulled into R memory, updating both the `tableOutput` and the `plotOutput` simultaneously. Meanwhile, an independent `httr2` pipeline reaches out to the public internet, fetches a JSON payload, parses it, and displays a random fact, protected perfectly by a `tryCatch` block in case the API goes down.

Extension Challenge:

1. Add a new `textInput` and an "Add Employee" `actionButton` to the UI.
2. Write an `observeEvent(input$add_btn, {...})` block in the server that executes an `INSERT INTO` SQL statement via `dbExecute()` to permanently add the new employee to the SQLite database.

ASSESSMENTS & INTERVIEW PREP

15+ Practice Questions

Test your fundamental knowledge of Web Development and Databases.

1. What HTTP verb is strictly used for fetching data from a server?
2. What does REST stand for in web architecture?
3. In a JSON string, what symbols represent an Array versus an Object?
4. What is the required function in `httr2` that physically executes the request across the internet?
5. What are the two mandatory components of a Shiny application?
6. Inside a Shiny server function, why do we call reactive variables with parentheses, like `my_data()`?
7. What does RDBMS stand for?
8. What is the difference between `DBI` and `RSQLite`?
9. Write the SQL command to fetch the `name` column from the `users` table.
10. What does CRUD stand for?
11. Which DBI function should you use for an `UPDATE` statement: `dbGetQuery` or `dbExecute`?
12. When using `dbplyr`, what function translates the lazy query into SQL and downloads the result to R?
13. What HTTP status code indicates a successful 'OK' response?
14. If an API requires a Bearer Token, what specific HTTP concept is this related to (Auth or CORS)?
15. Why is it dangerous to put a massive data-import function directly inside a Shiny `server` block?

IT STUDY HUB

20+ Viva Questions (With Answers)

These questions are designed for rapid-fire university viva or oral examinations.

Q1: Explain the Client-Server architecture.

Answer: The Client (browser or R script) requests data or services. The Server processes the request and returns a response.

Q2: What is the difference between HTTP GET and POST?

Answer: GET retrieves data from the server. POST submits new data to the server.

Q3: What is JSON?

Answer: JavaScript Object Notation. It is a lightweight, human-readable text format used globally for transmitting data across APIs.

Q4: Why use `httr2` instead of older packages?

Answer: It provides a highly robust, pipe-friendly interface (`|>`) and handles error routing and retries more safely.

Q5: What is the purpose of `resp_body_json()`?

Answer: It decodes the raw byte response from the server and converts the JSON payload into a native R list.

Q6: What does the Shiny UI do?

Answer: It constructs the frontend HTML webpage, organizing layout panels and collecting user inputs via widgets.

Q7: What does the Shiny Server do?

Answer: It acts as the backend engine, running the R code required to calculate statistics, filter data, and generate plots based on UI inputs.

Q8: What is Reactive Programming in Shiny?

Answer: A programming paradigm where outputs instantly and automatically re-calculate themselves whenever their underlying input variables change.

Q9: What is the difference between `input$x` and `reactive({ input$x })`?

Answer: `input$x` is a raw value. Wrapping it in `reactive({})` creates an intermediate, reusable cached expression that multiple outputs can rely on efficiently.

IT STUDY HUB

Q10: Why use SQLite?

Answer: It is a serverless, zero-configuration database stored as a single file, perfect for testing or lightweight R application backends.

Q11: What is the role of the DBI package?

Answer: It provides a universal frontend interface for database communication. You write one set of DBI code, and it works for MySQL, Postgres, or SQLite.

Q12: What does `dbGetQuery()` do?

Answer: It sends a SQL `SELECT` statement to the database and pulls the resulting data frame into R's memory.

Q13: What does `dbExecute()` do?

Answer: It sends actions that modify the database (INSERT, UPDATE, DELETE) and returns the number of rows affected, not a data frame.

Q14: What is `dbplyr`?

Answer: A backend for `dplyr` that automatically translates standard R dplyr code (like `filter`, `mutate`) into native SQL queries.

Q15: Why is `dbplyr` computationally efficient?

Answer: Because it executes the calculations lazily on the database server itself. R only downloads the final summarized results when `collect()` is called.

Q16: What is SQL Injection?

Answer: A hacking technique where malicious users type raw SQL commands into an input field (like a login box) to trick the database into executing them.

Q17: How do you prevent SQL Injection in R?

Answer: By using Parameterized Queries (binding variables safely via DBI) rather than pasting user strings directly into SQL code.

Q18: What is an API Authentication Token?

Answer: A secure, secret string passed in the HTTP Headers that proves your identity to the server so it grants access to private data.

IT STUDY HUB

Q19: What is CORS?

Answer: Cross-Origin Resource Sharing. A browser security mechanism that restricts a webpage from making API requests to a different domain than the one that served the webpage.

Q20: Why must you use `dbDisconnect()`?

Answer: To safely close the network connection. Leaving ghost connections open will eventually overwhelm the database server and cause a crash.

15+ Corporate Interview Questions

Prepare for advanced Full-Stack Data Science rounds with these architectural questions.

1. Walk me through the exact architectural sequence of events when an R script makes an `httr2` GET request to a REST API and converts the result to a dataframe.

2. Explain the fundamental difference between REST APIs returning JSON and traditional web servers returning HTML. Why do data scientists heavily prefer REST?

3. You are building a Shiny app that queries a 50GB PostgreSQL table. If you use `read.csv()`, your app crashes. Explain how `DBI` and `dbplyr` solve this problem.

4. Explain the concept of 'Lazy Evaluation' within the `dbplyr` framework. What specific function forces the evaluation to complete?

5. Contrast `dbGetQuery` with `dbExecute`. Provide a specific SQL example of when you would use each.

6. In Shiny, explain the lifecycle of a Reactive Expression. Why is it more computationally efficient to put heavy filtering logic inside a `reactive({})` block rather than directly inside a `renderPlot({})` block?

7. What is SQL Injection, and how does string-pasting (using `paste0` to build SQL queries) create catastrophic security vulnerabilities in Shiny applications?

8. Explain the difference between Authentication and Authorization in API security.

9. A public API returns a '429 Too Many Requests' HTTP status code. How do you architect your `httr2` pipeline to handle rate-limiting gracefully without crashing the script?

IT STUDY HUB

10. Compare the architecture of SQLite to a dedicated PostgreSQL server. When would you strictly choose PostgreSQL for a production R application?

11. You are parsing a complex, deeply nested JSON response using `jsonlite`. How do you conceptually flatten hierarchical JSON arrays into a Tidy rectangular dataframe?

12. What does CORS stand for, and why does a Shiny application hosted on shinyapps.io sometimes get blocked from fetching data from an external browser-based JavaScript API?

13. Walk me through the SQL CRUD acronym. Map each operation to its corresponding SQL command.

14. Why is it an absolute requirement to store API Keys and Database Passwords in an `.Renviron` file instead of directly in your `app.R` script?

15. Explain how you would architect a Shiny app to manage concurrent users. How does the scope of variables declared *outside* the `server` function differ from variables declared *inside* the `server` function?

IT STUDY HUB

MODULE WRAP-UP

Module Summary

In Module 11, you evolved from a Data Scientist into a Full-Stack R Developer. You learned how to bridge your R environment to the global internet, utilizing the HTTP protocol and `httr2` to seamlessly ingest JSON payloads from live REST APIs. You mastered the architecture of Relational Databases, using `DBI` to execute secure SQL CRUD operations, and `dbplyr` to effortlessly translate R logic into optimized database queries. Most importantly, you learned to deploy your analytical models to the world using the Shiny framework, utilizing reactive programming to construct living, dynamic web dashboards. Your R code is no longer confined to your local machine; it is now a fully integrated, secure web application.

IT STUDY HUB

Key Terms Dictionary

Term	Definition
HTTP	HyperText Transfer Protocol. The foundation of internet data communication.
REST API	An architectural style for APIs that uses standard HTTP methods (GET, POST).
JSON	JavaScript Object Notation. A universal, lightweight format for storing/ transporting data.
Shiny	An R package for building interactive web applications straight from R.
Reactivity	A programming paradigm where outputs automatically update when inputs change.
SQL	Structured Query Language. Used for managing data in relational databases.
CRUD	Create, Read, Update, Delete. The 4 basic operations of persistent storage.
DBI / dbplyr	Packages used to connect R to databases and translate dplyr code into SQL.

Quick Revision Checklist

- ✅ Do you know the difference between a GET request and a POST request?
- ✅ Can you confidently explain the difference between Shiny UI and Shiny Server?
- ✅ Do you understand why reactive variables require parentheses `()` when called?
- ✅ Do you know the difference between `dbGetQuery` (Read) and `dbExecute` (Write)?

- ✅ Do you understand why `collect()` is necessary at the end of a `dbplyr` pipeline?

Backend Coding Challenges

Challenge 1: The API Fetcher
Write a 3-line `httr2` pipeline that makes a GET request to `https://api.coindesk.com/v1/bpi/currentprice.json`, performs the request, and parses the JSON body.

Challenge 2: The Database Architect
Connect to an in-memory SQLite database. Use `dbExecute` to create a table: `CREATE TABLE users (id INTEGER, name TEXT)`. Then use `dbExecute` to `INSERT INTO users VALUES (1, 'Dithsa')`. Finally, use `dbGetQuery` to `SELECT *` from it.

Challenge 3: The Shiny Skeleton
Write the absolute minimum code required to launch a Shiny app: Load the library, define an empty `fluidPage` UI, define an empty `server` function, and run `shinyApp`.

IT STUD

WHAT COMES NEXT:

MODULE 12 — ADVANCED R, AUTOMATION, AI & PRODUCTION

You have built web apps and connected to databases. In the final module, we cover absolute enterprise-grade deployment: Dockerizing your R environments, integrating LLMs (ChatGPT) into your R scripts, and automating tasks globally via CI/CD pipelines.

IT STUD